

Implementing Machine Learning in Coded Mask Imaging of Gamma-Ray Sources

Ashoka University, Sonipat, Haryana

Department of Physics

Project Report

Submitted by:

Prashant Kumar Bai

prashant.bai_phd24@ashoka.edu.in

<https://github.com/prashant>

Project Advisor:

Prof. Dipankar Bhattacharya

Supervisor:

Prof. Suratna Das

Monsoon 2025

Contents

Abstract	2
1 Introduction	3
1.1 Motivation: The Need for Coded-Mask Imaging in High-Energy Astronomy . .	3
1.2 Basic Principle of Coded-Mask Imaging	4
1.3 Historical Development of Coded-Aperture Imaging	5
1.3.1 Early Ideas (1960s)	5
1.3.2 Uniformly Redundant Arrays (1970s): A Major Breakthrough	5
1.3.3 Application in Astronomy and Medical Science	6
1.4 Types of Mask Patterns and Their Properties	7
1.4.1 Random and Pseudo-Random Masks	7
1.4.2 Uniformly Redundant and Related Arrays	8
1.5 Image Reconstruction Method[1]	8
1.6 Advantages and Limitations of Coded-Mask Imaging	10
2 Designing mask pattern and constructing a shadow arrays library	11
2.1 The mask pattern	11
2.2 Simulation of shadow arrays	11
3 The Detector Plane Histogram	13
4 Image reconstruction and analysis	14
4.1 Richardson Lucy Bayesian Iteraion method [2]	14
4.1.1 Conceptual introduction to the Bayesian (EM) iteration method	14
4.1.2 Bayesian Iteration algorithm	15
4.2 Machine Learning approach	16
4.2.1 Polynomial Curve fitting [3]	16
4.2.2 Non-negative least square fitting	21
4.2.3 Ridge Regression and LASSO	22
4.2.4 Sequential Learning	24
4.2.5 Artificial Neural Networks	25
Conclusion	29

Abstract

Imaging in Gamma rays, the most energetic band of the electromagnetic spectrum, cannot be done by conventional means as it is not possible to construct reflective or refractive optics. What is employed instead is indirect imaging methods, of which Coded Mask imaging is an example. In this, a mask pattern is placed in front of an array of detectors. The light passes through the mask and casts different shadow patterns over the detector plane depending on the angle it makes with the perpendicular to the mask. A detector plane histogram (DPH) is created depending upon the angle and the intensity of the sources which is then sent to the ground station to analyze and construct the image.

However, the data from the satellite can only be received by the ground station when it enters the field of detection of the ground based radio receiver. Moreover, it takes a long time to reconstruct images because it's an indirect, computationally intensive process involving complex deconvolution/decoding algorithms (like MLEM or EM-based methods) that solve an ill-posed problem, correlating noisy detector data with the mask pattern to infer the original source, which requires many iterations or high processing power to separate signal from background noise and achieve reasonable resolution. Due to all these factors it's nearly impossible to do imaging of a fast moving or a transient source like a pulsar.

To overcome this obstacle, we've made an effort to develop and train a neural network model that will be more efficient and faster than the former method. This model, if installed on a micro-chip and sent to space with the satellite, can instantly construct the image and detect the location of a γ ray source, enabling the satellite to orient towards the source's new location and communicate other nearby satellites in real time.

Chapter 1

Introduction

1.1 Motivation: The Need for Coded-Mask Imaging in High-Energy Astronomy

Imaging is one of the most fundamental tools in observational science. In optical astronomy, high-quality images are routinely obtained using lenses and mirrors that focus incoming light onto a detector. However, this approach becomes increasingly difficult and eventually impossible as one moves to higher photon energies (>15 keV), such as X-rays and gamma rays. At these energies, conventional refractive optics do not work because the refractive index of most materials is extremely close to unity, and reflective optics require extremely shallow grazing incidence angles, which severely restricts the usable energy range and field of view.

While grazing-incidence mirrors have been successfully developed for soft X-rays, they become impractical for hard X-rays (10-100 keV) and soft gamma-rays (100 keV–10 MeV) due to both physical and technological limitations. As a result, early high-energy astronomy faced a fundamental problem: how to determine not only the intensity but also the direction of arrival of high-energy photons without using conventional focusing optics.

The simplest solution is a pinhole camera, where a single small aperture projects an image onto a detector. While conceptually simple, this method suffers from extremely poor photon throughput: most of the incoming radiation is blocked, which leads to low sensitivity and poor signal-to-noise ratio, especially for faint cosmic sources.

Coded mask/aperture imaging systems or Coded Mask Instruments (**CMI**) was developed as a powerful generalization of the pinhole camera. Instead of a single aperture, a patterned mask consisting of many transparent and opaque elements is placed in front of a position-sensitive detector. This mask casts a shadow pattern on the detector plane. Because the mask pattern is known, the recorded shadow can be mathematically processed to reconstruct the original sky intensity distribution. In this way, coded-mask systems achieve much higher photon throughput while still preserving directional information.

This idea — sacrificing direct imaging in favor of computational reconstruction —

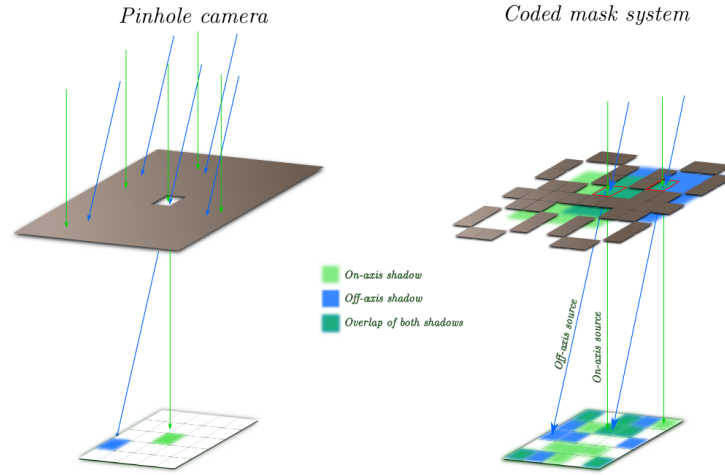


Figure 1.1: Left: a pinhole camera or a single-open-cell coded-mask imager. The detector plane registers an image of the source field with relatively low significance peaks, since the detecting area corresponds to only one mask element. Right: a coded-mask imager. The detector plane registers an overlapping set of multiple images, each set associated with one point source.

represents one of the earliest and most successful examples of what is now broadly called *computational imaging*.

However, at energies higher than a few MeV the mask material becomes more and more transparent, the Compton scattering dominant and the geometrical spatial modulation, based on the photoelectric absorption process, less efficient. Absorption increases again at energies >10 MeV thanks to the pair-production effect, but there it is more efficient to use the intrinsic directional properties of this interaction on the few detected photons rather than the collective effect of shadow projection by many rays in order to measure the source direction. X/gamma-ray astronomy is also the domain where the high and variable background becomes dominant over the source contributions, which drastically limits the performance of standard on/off monitoring techniques and where the simultaneous measurement of source and background is crucial even for the simple source detection.

1.2 Basic Principle of Coded-Mask Imaging

A coded-mask imaging system consists of three main components:

1. A mask with a known pattern of opaque and transparent elements,
2. A detector plane capable of recording the spatial distribution of incoming photons,
3. A known separation distance between the mask and the detector.

Each point source in the sky produces a shifted shadow of the mask pattern on the detector. If multiple sources are present, the detector records a linear superposition of these shadow

patterns, weighted by the source intensities. Mathematically, the detector plane image can be described as the convolution of the sky brightness distribution with the mask transmission function, plus background and noise.

Reconstruction of the sky image is therefore an inverse problem: given the detector image and the known mask pattern, one attempts to recover the original sky intensity distribution. Depending on the mask design and the reconstruction method, this can be done by:

- Cross-correlation with the mask pattern,
- Direct linear deconvolution,
- Maximum likelihood or Bayesian methods,
- Iterative and regularized inversion techniques.

The quality of the reconstruction depends critically on the mathematical properties of the mask pattern, the signal-to-noise ratio, background modeling, and detector effects.

1.3 Historical Development of Coded-Aperture Imaging

1.3.1 Early Ideas (1960s)

The basic concept of replacing a single pinhole with a pattern of multiple apertures emerged in the 1960s. One of the earliest and most influential proposals was by Robert H. Dicke, who introduced the idea of a “scatter-hole camera” for high-energy radiation. Similar ideas were also proposed independently by Ables and others around the same period.

The key realization was that, although multiple apertures mix information from different directions, this mixing is not random: if the aperture pattern is known, the mixing can be mathematically inverted. These early systems often used random or semi-random hole patterns, and reconstruction was performed using correlation techniques.

While the basic feasibility was demonstrated, early implementations suffered from strong artifacts, side-lobes, and noise amplification, largely due to suboptimal mask designs and limited computational resources.

1.3.2 Uniformly Redundant Arrays (1970s): A Major Breakthrough

A major breakthrough came in 1978 when **Edward Fenimore** and **Thomas Cannon** introduced the concept of the **Uniformly Redundant Array (URA)**. URA masks are specially designed binary patterns whose autocorrelation function has nearly ideal properties: a sharp central peak and a flat, minimal sidelobe structure.

The importance of URAs lies in the fact that, for such masks, simple correlation of the detector image with the mask pattern yields a reconstructed image with:

- A well-defined point spread function,
- Strong suppression of ghost images and sidelobes,
- Efficient and stable reconstruction.

This development transformed coded-mask imaging from a somewhat ad-hoc technique into a rigorously designed and mathematically grounded imaging method.

Following URAs, several related designs were developed, including:

- Modified Uniformly Redundant Arrays (MURA),
- Hexagonal URAs (HURA),
- Other cyclic and optimized binary patterns.

These designs offered improved flexibility in mask geometry, field of view, and mechanical implementation. An example of both URA and MURA based mask is shown in fig. 1.2

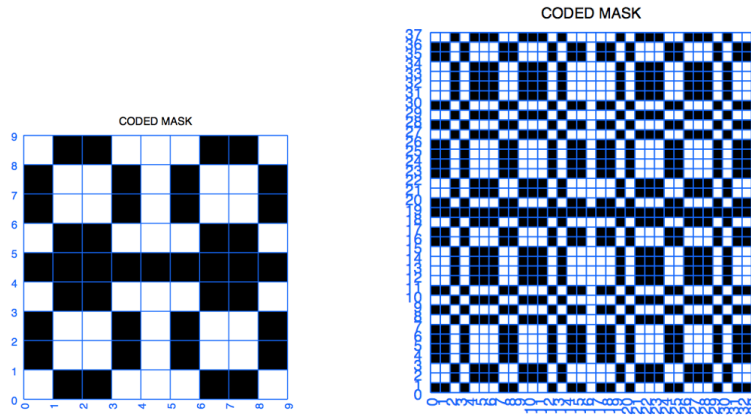


Figure 1.2: Left: A 5x5 MURA-based mask. Right: A 19x17 URA-based mask.

1.3.3 Application in Astronomy and Medical Science

Coded masks were conceived in the 1970s-1980s and employed successfully in the past 40-30 years in high-energy astronomy, on balloon-borne instruments first and then onboard space missions like Spacelab 2, GRANAT, and BeppoSAX. They have been chosen as imaging systems for experiments on a number of major missions presently in operation, the European INTEGRAL, the American Swift and the **Indian ASTROSAT**[2], and for some future projects like the Chinese-French SVOM mission. Today the NuSTAR and the Hitomi missions have successfully pushed up to 80 keV the technique of grazing incidence X-ray mirrors. However the limited field of view (few arcmin) achieved by these telescopes and the variability of the sky at these energies make the coded mask systems still the best options to search for bright transient or variable events in wide field of views.

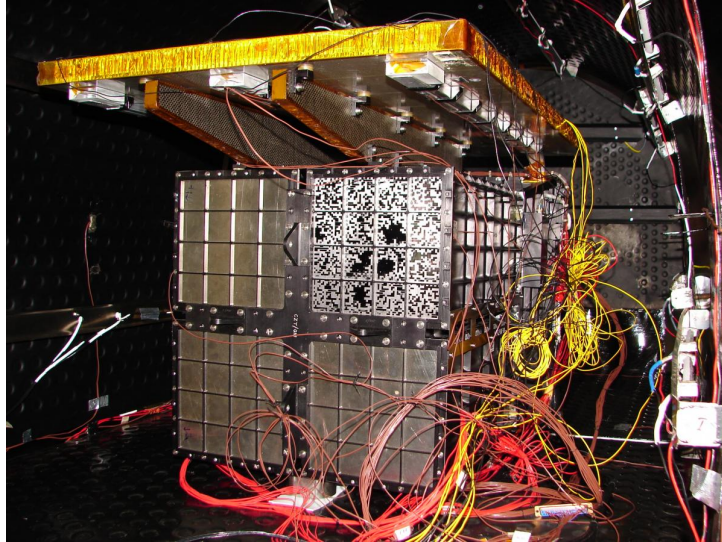


Figure 1.3: Mask plate of ASTROSAT CZTI under development

Coded aperture systems have been employed also in medicine and in monitoring nuclear plants and implementations in nuclear security programs are also envisaged. Even if basic concepts are still valid for these systems, certain conditions, specific to gamma-ray astronomy, can be relaxed (e.g., source at infinity, high level and variable background, etc.), and therefore designs and data analysis for CMI for terrestrial studies can take a very different form. In particular for close sources (the so-called near-field condition), the system can actually provide three-dimensional imaging because of the intrinsic enlarging effect of shadow projection as the source distance decreases. This interesting property is not applicable in astronomy and we will not discuss it here.

1.4 Types of Mask Patterns and Their Properties

The choice of mask pattern is one of the most critical design decisions in a coded-mask imaging system.

1.4.1 Random and Pseudo-Random Masks

Early systems often used random patterns. While easy to generate, such masks generally have:

- Non-ideal autocorrelation properties,
- Strong sidelobes,
- More reconstruction artifacts.

However, with modern reconstruction techniques and sufficient signal-to-noise, optimized random masks are still sometimes used, especially when mechanical or design constraints dominate.

1.4.2 Uniformly Redundant and Related Arrays

URA, MURA, and related designs are based on number-theoretic constructions that guarantee:

- Near-delta-function autocorrelation,
- Uniform response over the fully coded field of view,
- Efficient and stable reconstruction using correlation methods.

These masks are widely used in astronomical instruments due to their predictable and well-controlled imaging performance.

Trade-offs in Mask Design

In practice, mask design must balance several competing factors:

- Angular resolution (set by mask element size and mask-detector distance),
- Field of view (fully coded vs partially coded regions),
- Mechanical strength and manufacturing constraints,
- Transparency at different photon energies,
- Background and noise considerations.

1.5 Image Reconstruction Method[1]

The intensity of each skybin can be recovered by a matching process. If we represent the mask basic pattern by an array M_{ij} whose elements are 1 for the closed cells and 0 for the open ones, what gets imprinted in the detector plane can be described mathematically as a *correlation*:

$$D = S \otimes M + B \quad (1.1)$$

where $\otimes$ is the correlation operator. In a digital implementation, this can be written as

$$D_{ij} = \sum_{k=0}^{p-1} \sum_{l=0}^{q-1} S(k, l) M((k+i) \bmod p, (l+j) \bmod q) + B_{ij}; i = 0 \dots p-1, j = 0 \dots q-1 \quad (1.2)$$

where the elements of S are the intensities for sources in each skybin (counts per cell area) multiplied by the $\cos\theta$ factor, and B is an aperture-independent (instrumental) background (also in counts/cell).

In order to obtain S from D , several approaches and algorithms have been discussed in the literature and used in specific instruments with specific detector systems and mask

patterns. A widely used reconstruction method is the correlation of the recorded data with a suitable decoding function that in general mimics the mask pattern. In these cases Fast Fourier Transforms (FFT) have been extensively used to speed-up the reconstruction process, since the correlation operation in Fourier space becomes a multiplication. In what follows, the focus is on the properties of the reconstructed images rather than on the particular procedure to acquire it. Since $D = S \otimes M + B$,

$$D \otimes G = (S \otimes M) \otimes G + B \otimes G = S \otimes (M \otimes G) + B \otimes G \quad (1.3)$$

where G is the so-called *decoding function*. If $M \otimes G$ is a 2D discrete δ -function, the reconstructed image $\hat{S}$ is given by

$$\hat{S} = D \otimes G - B \otimes G, \quad (1.4)$$

Therefore, the intensities of each skybin (the image) are obtained by a cross-correlation of the count distribution in the detector plane with the decoding function and the image has no artifacts. If the background is uniform, the second term on the right side of equation (1.4) just adds a DC-level to the reconstructed image. If the background is known by an independent measurement, $\hat{S} = (D - B) \otimes G$. In both cases, the source map is reconstructed “perfectly” in the sense that it is free of intrinsic noise.

For URA-type masks, this already produces a good approximation to the true sky image. However, real instruments introduce complications:

- Detector non-uniformities,
- Dead pixels and gaps,
- Energy-dependent effects,
- Finite statistics.

Therefore, modern coded-mask systems often use:

- Maximum likelihood or maximum entropy methods,
- Iterative reconstruction techniques,
- Regularization and sparsity constraints,
- Forward-modeling approaches where the detector image is fit directly.

From a mathematical point of view, coded-mask imaging is a linear inverse problem, and its solution quality is limited by conditioning, noise, and model accuracy.

1.6 Advantages and Limitations of Coded-Mask Imaging

Advantages:

- High throughput compared to a pinhole camera,
- No need for focusing optics,
- Very large field of view possible,
- Excellent for monitoring transient and variable sources,
- Proven, robust technology for space missions.

Limitations:

- High background acceptance compared to focusing telescopes,
- Reconstruction artifacts and sidelobes,
- Angular resolution limited by geometry,
- Complex calibration and response modeling,
- Computationally intensive reconstruction for large systems.

Thus, coded-mask imaging is not a replacement for focusing optics, but rather a complementary technique, especially powerful in energy ranges where focusing is impractical.

Position of the present work

In this context, the present project focuses on the simulation, modeling, and reconstruction of coded-mask imaging data, with particular emphasis on understanding the forward model (shadow formation) and the inverse problem (image reconstruction). By studying controlled simulated data and reconstruction algorithms, deeper insight is gained into the strengths, limitations, and practical challenges of coded-mask imaging systems.

Chapter 2

Designing mask pattern and constructing a shadow arrays library

The algorithm to do the shadow arrays simulation has been written in Fortran 90 in Vim editor. The source code has been compiled using gfortran compiler. The DPH creation and the Bayesian iteration to reconstruct its image has also been done in the same language in a Linux OS.

2.1 The mask pattern

We design a 16x16 mask plate pattern keeping in mind its rigidity to withstand the vibrations and shock-waves that the satellite on which it is mounted experiences. We keep half of the squares in the plate open. Though the mask plate produces a unique shadow pattern for each sky location, it is still not an ideal one and is not an URA (refer 1.3.2). This pattern is then converted to a 16x16 matrix, say M consisting of only 1s and 0s where 1s belong to the closed cells and 0s belong to the open ones. This matrix serves as the input of the shadow arrays simulation algorithm. The pattern along with its readable matrix form is shown in Fig. 2.1

2.2 Simulation of shadow arrays

The detector plane located behind the mask plate is also a 16x16 array of pixels. It detects the average count of photons by each pixel. The distance, H between the mask and the detector plane is 600mm. The side length, L of the plane is 40mm. So each pixel is $2.5mm^2(L/16)$. Let j_x be the position of an illuminated pixel along x-axis caused by light coming from an open square in the mask. Then it can take total 31 values from $[-15, 15]$. A similar variable j_y can be assigned to locate the position of an illuminated pixel along y-axis which has the same range. But unlike the usual convention for j_x , j_y goes from -15 to 15 along negative y direction. So, our detector can detect signal from a total of 961 sky locations (31x31).

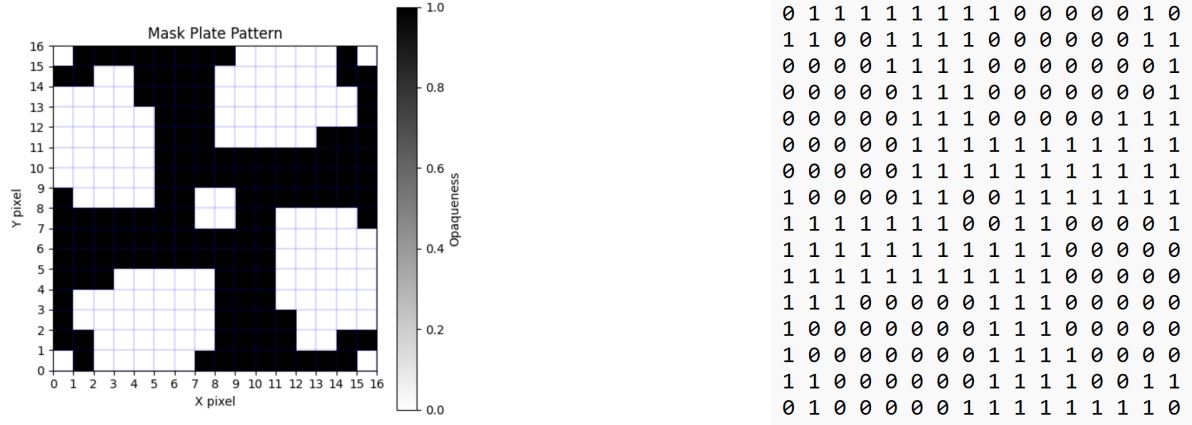


Figure 2.1: Left: The mask pattern. The cells in black are opaque and those in white are transparent to the γ -rays. Right: The mask pattern matrix. 1s correspond to the closed cells and 0s to the open ones

Based on these information we can calculate the following results:

$$\begin{aligned}
 i &= (j_x + 15) + 31 * (j_y + 15), \\
 d_x &= j_x * 2.5mm, d_y = -j_y * 2.5mm, \\
 \theta_x &= \arctan(d_x/H), \theta_y = \arctan(d_y/H),
 \end{aligned}$$

where the index i denotes the position of the source in the sky. It takes values from 0 to 960. d_x and d_y are the distances travelled by the point source with respect to the overhead position along x and y axis respectively and θ_x and θ_y are the angles made by the light ray with respect to the perpendicular to the mask plate. We design an algorithm that creates the shadow patterns, S_{ik} for all the respective sky sources i of unit intensity. Here k denotes the pixel index from 0 to 255. These shadow patterns are stored in a text file in the form of arrays which will be used to construct the image from a given detector plane histogram (DPH). Each shadow array contains only 1s and 0s as their elements where 1s correspond to illuminated pixels and 0s to dark ones.

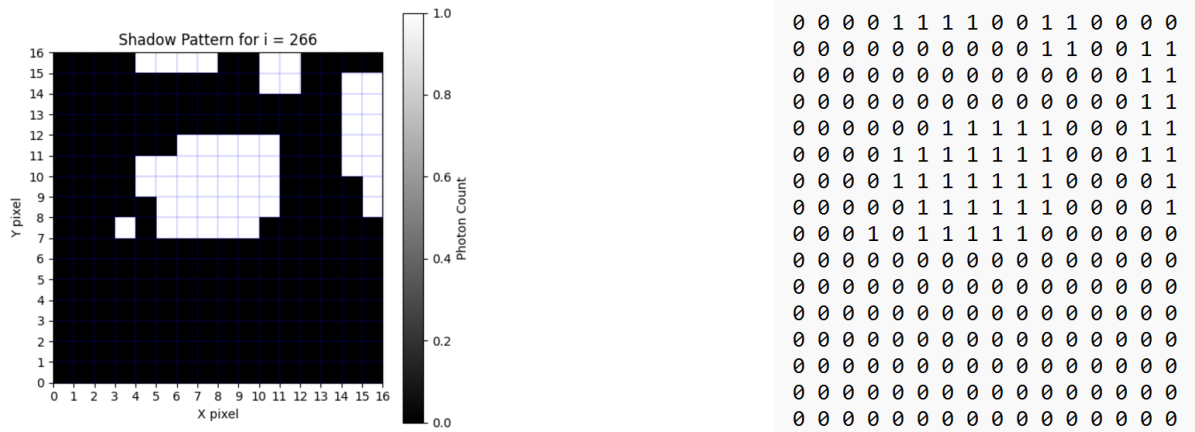


Figure 2.2: $i = 266$, $(j_x, j_y) = (3, 7)$, $(d_x, d_y) = (7.5, 17.5)$, $(\theta_x, \theta_y) = (0.71616^\circ, 1.67065^\circ)$

Chapter 3

The Detector Plane Histogram

The given dph is $D_k^{obs} = 30 * S_{50} + 15 * S_{512}$. So we know that the light comes from sky location 50 and 512 with intensities $30 \text{ cm}^{-2} \text{ s}^{-1}$ and $15 \text{ cm}^{-2} \text{ s}^{-1}$ respectively. The figure of the DPH is shown below. We will reconstruct the image from this same DPH in all the methods that we are going to use. However, we have to keep in mind that the DPH is not clean. It has some noise added to it.

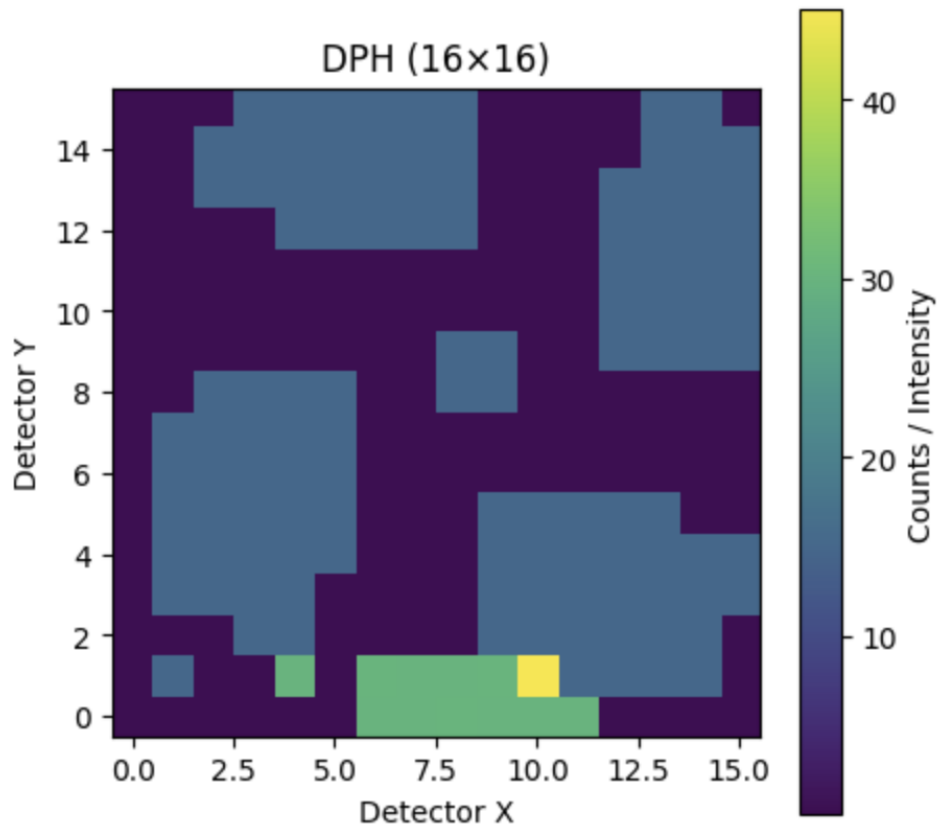


Figure 3.1: Colour coded plotting of DPH.

Chapter 4

Image reconstruction and analysis

4.1 Richardson Lucy Bayesian Iteraion method [2]

4.1.1 Conceptual introduction to the Bayesian (EM) iteration method

The image reconstruction problem in coded mask imaging is an inverse problem: the observed DPH is related to the true sky intensity distribution through a linear forward model involving the mask response (shadow patterns). However, this inverse problem is ill-posed and ill-conditioned, since many different sky intensity distributions can produce very similar detector responses, and the shadow patterns are not orthogonal.

Moreover, the measured detector counts are governed by Poisson statistics, because the data consist of photon counts. Therefore, the correct statistical approach is not ordinary least squares, but maximum likelihood estimation under Poisson noise.

The Bayesian iteration method used here is based on **maximum likelihood (ML) estimation** of the sky intensity distribution assuming Poisson-distributed detector counts. This leads to a likelihood function of the form

$$\mathcal{L}(I) = \prod_{j=1}^{N_{\text{det}}} \frac{M_j(I)^{D_j} e^{-M_j(I)}}{D_j!} \quad (4.1)$$

where D_j is the observed detector count in pixel j , and $M_j(I)$ is the model prediction obtained by projecting the sky intensity distribution I through the shadow patterns

Maximizing this likelihood is equivalent to minimizing the corresponding negative log-likelihood. This optimization problem is solved using an **Expectation–Maximization (EM)** type iterative scheme, which leads to a multiplicative update rule for the sky intensities. This update rule has several important properties:

- It preserves positivity of the sky intensities automatically.
- It conserves total flux (normalization) under appropriate conditions.

- It monotonically increases the likelihood at each iteration.
- It is well suited for Poisson-distributed data.

In each iteration, a model detector plane histogram (MDPH) is first computed from the current estimate of the sky image. Then a correction factor is computed by comparing the observed DPH to the model DPH, and the sky intensities are updated multiplicatively using the back-projection of this ratio through the shadow patterns. The iteration is repeated until a suitable convergence criterion is satisfied.

4.1.2 Bayesian Iteration algorithm

We first construct the MDPH for the initial sky image, $D_k^m = \sum_{i=0}^{960} I_i S_{ik}$, where I_i is the initial sky image array which contains the intensity of the sky locations; consisting of only 1s in every location. We decide a tolerance value, $\epsilon = 10^{-5}$. The actual image corresponding to the DPH is reconstructed through the following steps:

Step1: The correction factor C_i for each shadow array is calculated by the formula. All these factors are stored in an array, say C

$$C_i = \sum_{k=0}^{255} \frac{D_k^{obs}}{D_k^m} S_{ik} \quad (4.2)$$

Step2: The image array is updated by

$$I_i^{new} = C_i * I_i^{old} \quad (4.3)$$

Step3: A threshold value is set for the new sky intensities by the formula: $threshold = 3 * rms + mean$, where rms is the root mean square value or standard deviation of the intensities in the array and mean is the average of those intensities. Now for those new I_i values which are greater than the threshold value, $|1 - C_i|$ is calculated and checked whether the condition: $max|1 - C_i| \leq \epsilon$ is satisfied. This selective Bayesian iteration is necessary, otherwise it would take an exponential amount of iterations to meet the condition as the presence of negligible intensities slows down the rate of convergence of the iteration.

If the condition is not satisfied a new MDPH is constructed using the new image array and the above steps are reiterated till the condition is satisfied. **The DPH, MDPH, shadow arrays and the image array are all normalized before using in every iteration.**

It takes an average of 900 iterations to reconstruct the image. The intensity at source "50" comes out to be 2.99636708e+01 and at "512" it is 1.48824392e+01 which are significantly accurate to the actual values. This shows that we successfully managed to recover the image

from the given noisy DPH. The plot of intensities v/s sky locations is shown in Fig. 4.1

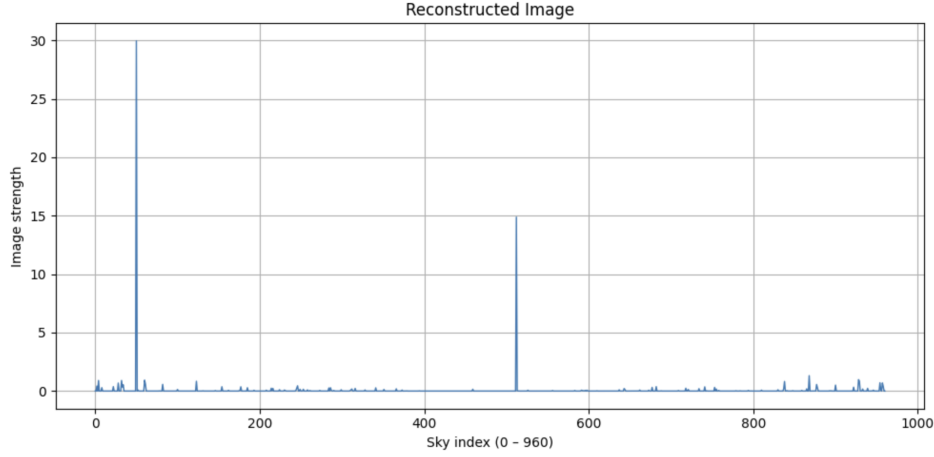


Figure 4.1: Plot showing intensity of sky locations reconstructed through Bayesian iterations

4.2 Machine Learning approach

We have used tools from the `sklearn.linear_model` package in Python to do the ML algorithm. We've built our Neural Network model in the CPU version of Tensorflow engine using Keras.

4.2.1 Polynomial Curve fitting [3]

Before proceeding with the machine learning tools, we need to know the mathematics behind them to have a better understanding of which tool or algorithm can give us satisfactory results in reconstructing the image. Unlike Bayesian curve fitting the tools that we are going to use from the `scikit.learn` module use polynomial curve fitting. Let us understand this concept with the help of a simple example.

Suppose we have a real-valued input and target vector; $\vec{x}$ and $\vec{t}$ respectively. They are related to each other by the relation $t = \sin(2\pi x)$. As we know the relation, we can compare the model curve with the actual curve to see how precisely it is fitting. We generate 10 training data points from the curve with random noise added to each of the target values as shown in the Fig. 4.2

$$\vec{x} = (x_1, x_2, \dots, x_{10}) \quad (4.4)$$

$$\vec{t} = (t_1, t_2, \dots, t_{10}) \quad (4.5)$$

We shall fit the data using a polynomial function of the form

$$y(x, \vec{w}) = w_0 + w_1x + w_2x^2 + \dots + w_Mx^M \quad (4.6)$$

where $w_0, w_1, \dots, w_M$ are components of $\vec{w}$. The polynomial is not a linear function of x , but

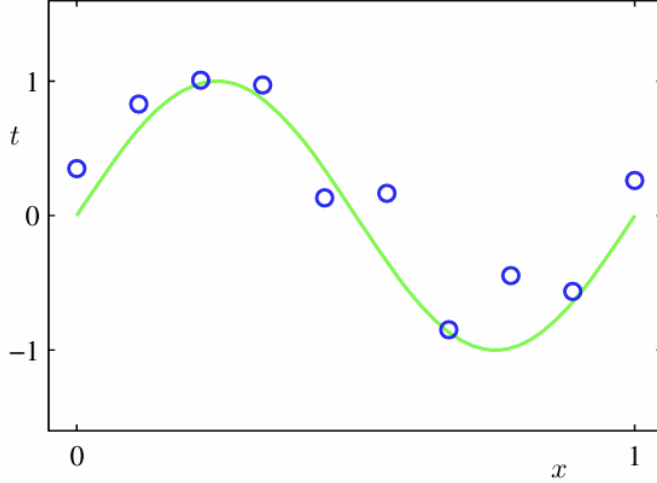


Figure 4.2: Plot of a training data set of $N = 10$ points, shown as blue circles, each comprising an observation of the input variable x along with the corresponding target variable t . The green curve shows the function $\sin(2\pi x)$ used to generate the data. Our goal is to predict the value of t for some new value of x , without knowledge of the green curve.

it is linear for $\vec{w}$. That's why it is known as a *linear model*. The values of these coefficients of x will be given by fitting $y(x, \vec{w})$ to the data points. We also need to minimize the error between the fitting and the data values, i.e, between $y(x_n, \vec{w})$ and t_n . For this we generate an error function which will be a sum over squares of differences between real output and predicted output, $E(\vec{w})$ and minimize it:

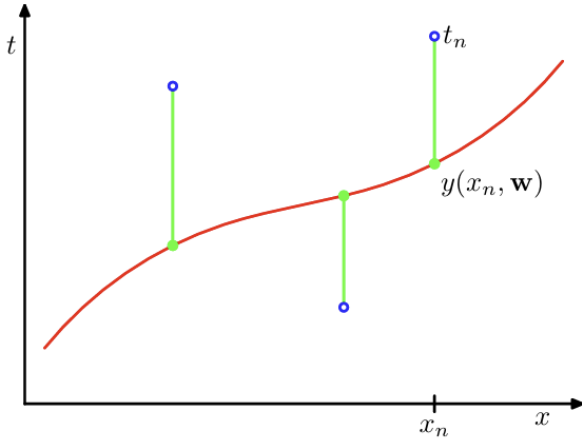


Figure 4.3: The error function (4.7) corresponds to (one half of) the sum of the squares of the displacements (shown by the vertical green bars) of each data point from the function $y(x, \vec{w})$.

$$E(\vec{w}) = \frac{1}{2} \sum_{n=1}^N \{y(x_n, \vec{w}) - t_n\}^2 \quad (4.7)$$

The geometrical representation of the sum-of-squares error function is illustrated in Fig. 4.3. The error function is a quadratic function of $\vec{w}$. So $\frac{d}{d\vec{w}} E(\vec{w})$ will be linear. In order to minimize it,

$$\frac{d}{d\vec{w}} E(\vec{w}) = \sum_{n=1}^N \{[y(x_n, \vec{w}) - t_n] y'(x_n, \vec{w})\} = 0 \quad (4.8)$$

The above equation will have only one solution, say $\vec{w}^*$, which can be found in closed form. The resulting polynomial is given by the function $y(x, \vec{w}^*)$.

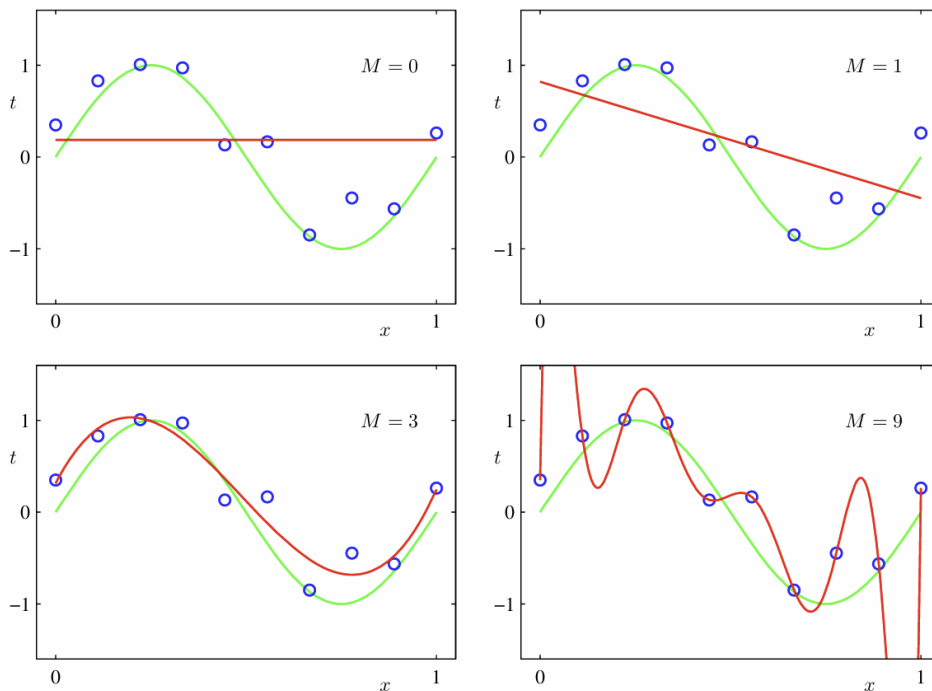


Figure 4.4: Plots of polynomials having various orders M , shown as red curves, fitted to the data set shown in Fig. 4.2. Plots for $M = 0, 1$ show poor fitting, $M = 3$ shows good fitting. Whereas $M = 9$ shows over-fitting.

The next task is to determine M . From the Fig. 4.4 we can see that $M = 3$ gives the best fit but it does not let y to pass through all the data sets. When we go to a much higher order polynomial ($M = 9$), we obtain an excellent fit to the training data. In fact, the polynomial passes exactly through each data point and $E(\vec{w}) = 0$. However, the fitted curve oscillates wildly and gives a very poor representation of the function $\sin(2\pi x)$. This latter behaviour is known as *over-fitting*, but it gives a good quantitative insight on the generalization, i.e., how t depends on x . To obtain more quantitative insight into the dependence of the generalization performance on M , we generate 100 separate test data points by the same procedure as for the training data points but with new choices of random noise values. For each M , we can evaluate $E(\vec{w}^*)$ for both test as well as training data sets.

It is sometimes more convenient to use the RMS error defined by

$$E_{RMS} = \sqrt{2E(\vec{w}^*)/N} \quad (4.9)$$

in which the division by N allows us to compare different sizes of data sets on an equal footing, and the square root ensures that E_{RMS} is measured on the same scale as the target variable t .

As we increase M , the magnitude of coefficients; $w_0^*, w_1^*, \dots, w_9^*$ get larger. For $M = 9$ polynomial, the coefficients have become finely tuned to the data by developing large positive and negative values (refer Table 4.6) so that the corresponding polynomial function matches each of the data points exactly, but between these points the function exhibits large oscillations.

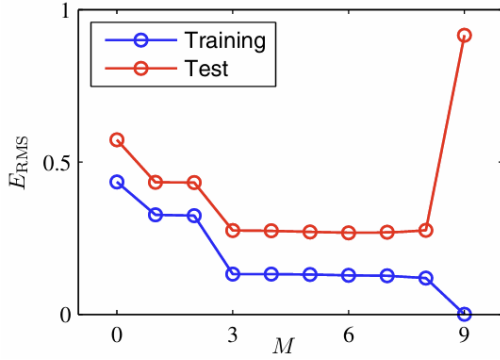


Figure 4.5: Graphs of the root-mean-square error, defined by (4.9), evaluated on the training set and on an independent test set for various values of M .

The more flexible polynomials with larger values of M are becoming increasingly tuned to the random noise on the target values.

Figure 4.6: Table of the coefficients w^* for polynomials of various order. The typical magnitude of the coefficients increases dramatically as the order of the polynomial increases.

	$M = 0$	$M = 1$	$M = 6$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

The problem of over fitting can be greatly reduced if we increase the number of data points. The more the number of data points we have, the more complex model we can afford to fit. We can see this in Fig. 4.7

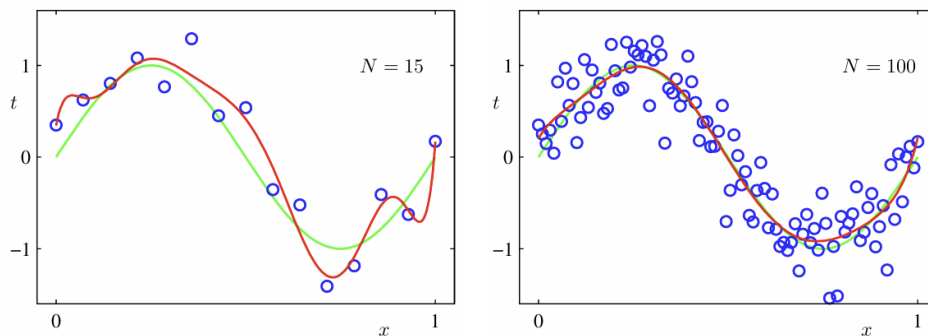


Figure 4.7: Plots of the solutions obtained by minimizing the sum-of-squares error function using the $M = 9$ polynomial for $N = 15$ data points (left plot) and $N = 100$ data points (right plot). We see that increasing the size of the data set reduces the over-fitting problem.

However, the problem still persists; how to find the best fit complex and flexible models using a very small data set. One way to do this is by introducing *regularization* term, which

suppresses the coefficients' magnitude and inhibits the model's violent oscillations. Then, the modified error function becomes

$$\tilde{E}(\vec{w}) = \frac{1}{2} \sum_{n=0}^N \{y(x_n, \vec{w}) - t_n\}^2 + \frac{\lambda}{2} \|\vec{w}\|^2 \quad (4.10)$$

where $\|\vec{w}\|^2 = \vec{w}^T \cdot \vec{w} = w_0^2 + w_1^2 + w_2^2 + \dots + w_M^2$. Again this error function can be minimized in exactly the same manner as the previous one. This particular case of quadratic regularizer is called *Ridge Regression*. We will learn more about this method in section 4.2.3. In the context of neural networks, this approach is known as *weight decay*.

However, λ has a sweet spot. It can't be increased too much, neither be decreased, else the complex model would not fit satisfactorily. If we decrease it too much, the error function would behave similar to the unregularized one and we would get large magnitudes of w_m s which would make the model too oscillatory. If we increase λ too much, then also the model would not fit as shown in Fig. 4.8 as the coefficients would be suppressed so much that most of them would approach to zero. The value of the coefficients for all the three cases have been shown in Table 4.9

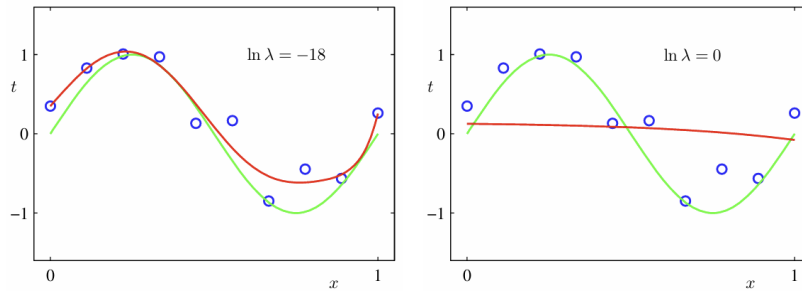


Figure 4.8: Plots of $M = 9$ polynomials fitted to the data set shown in Fig. 4.2 using the regularized error function (4.10) for two values of the regularization parameter λ corresponding to $\ln \lambda = -18$ and $\ln \lambda = 0$. The case of no regularizer, i.e., $\lambda = 0$, corresponding to $\ln \lambda = -\infty$, is shown at the bottom right of Fig. 4.4

	$\ln \lambda = -\infty$	$\ln \lambda = -18$	$\ln \lambda = 0$
w_0^*	0.35	0.35	0.13
w_1^*	232.37	4.74	-0.05
w_2^*	-5321.83	-0.77	-0.06
w_3^*	48568.31	-31.97	-0.05
w_4^*	-231639.30	-3.89	-0.03
w_5^*	640042.26	55.28	-0.02
w_6^*	-1061800.52	41.32	-0.01
w_7^*	1042400.18	-45.95	-0.00
w_8^*	-557682.99	-91.53	0.00
w_9^*	125201.43	72.68	0.01

Figure 4.9: Table of the coefficients for $M = 9$ polynomials with various values for the regularization parameter λ . Note that $\ln \lambda = -\infty$ corresponds to a model with no regularization, i.e., to the graph at the bottom right in Figure 4.4. We see that, as the value of λ increases, the typical magnitude of the coefficients gets smaller.

4.2.2 Non-negative least square fitting

Since our problem of image reconstruction from a given DPH is a linear one as

$$d_{256 \times 1} \simeq A_{256 \times 961} c_{961 \times 1}, \quad (4.11)$$

where d is the DPH column matrix, A contains all the 961 shadow arrays in column form and c is the sky intensities column matrix, we are going to use methods of *Linear Regression* to solve this whose mathematics we learned in the previous section. Our linear model will be of the form:

$$y(\vec{x}, \vec{w}) = w_0 + w_1 x_1 + w_2 x_2 + \dots + w_D x_D \quad (4.12)$$

Here $\vec{x} = (x_1, x_2, \dots, x_D)$ represents all the shadow arrays and $\vec{w} = (w_0, w_1, w_2, \dots, w_D)$ represents the sky intensities, except the w_0 which contributes to the background or noise in the DPH.

We will begin first by the method of non-negative least square (NNLS) fitting. For this we import *nls* from *scipy.optimize*. What we are going to do is $\min_{c \geq 0} \|Ac - d\|^2$, i.e., find c for which the difference between the linear combination of shadow patterns with suitable coefficients (sky intensities) and the DPH is minimum. This method is similar to the least square fitting method where we minimize the sum-of-squares error function, $E(\vec{w})$ (4.7). But here our method is subjected to the condition that all the elements in c will be greater than or equal to zero, since sky source intensities cannot be negative.

The plot of the reconstructed image for both clean as well as noisy DPH with a list of significant signal strengths is shown in Fig. 4.10 and Fig. 4.11 respectively.

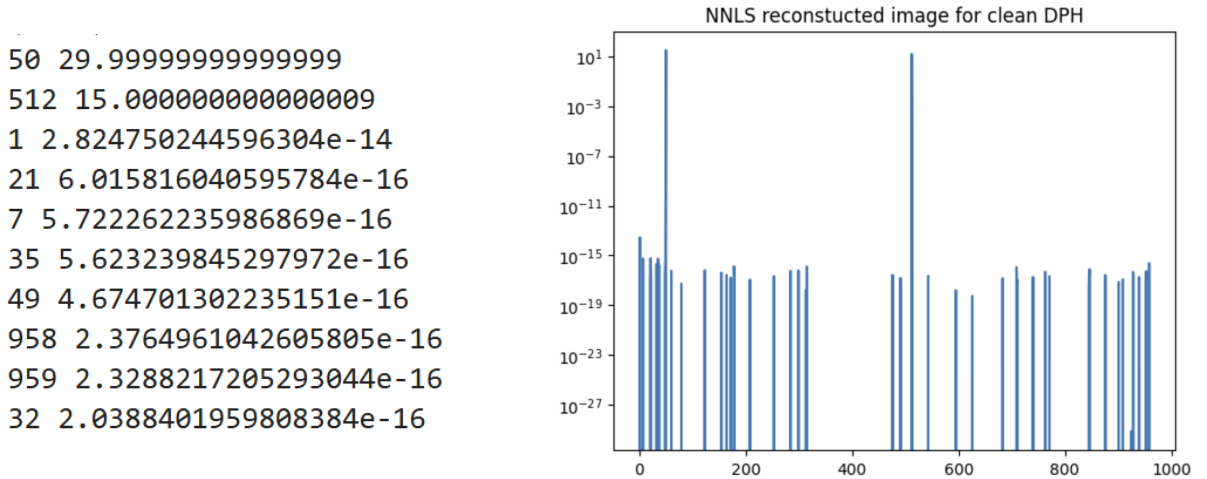


Figure 4.10: Left: List of signal strengths. The first column lists out the sources and the second one their respective strengths. Right: Plot of intensity v/s sky location

We can see that this method gives a good reconstruction of image for the clean DPH, but fails to do it for the noisy one. The sources where there is no signal are also showing significant

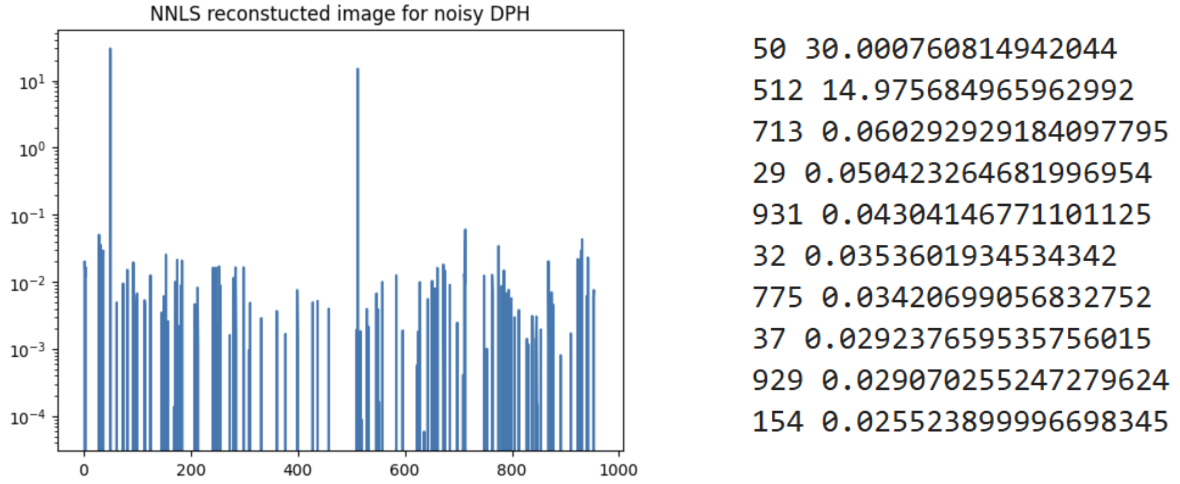


Figure 4.11: Left: Plot of intensity v/s sky location for noisy DPH. Right: List of signal strengths with their location

strength of the order $1e-2$ which cannot be ignored. We need to build an algorithm which could be more practical, i.e., filter out noise and give image of reasonable resolution. For this we need to add regularization to suppress the intensity of sources where there is no light.

4.2.3 Ridge Regression and LASSO

The methods which we are going to use in this section are based on the principle of regularized least square fitting which we learned in the section 4.2.1. The general form of error function can be written as

$$E(\vec{w}) = \frac{1}{2} \sum_{n=1}^N \{t_n - y(\vec{x}_n, \vec{w})\}^2 + \frac{\lambda}{2} \sum_{j=1}^M |w_j|^q \quad (4.13)$$

where the second term acts as a regularizer.

For $q = 2$, $E(\vec{w})$ is called **Ridge Regression**. It is also known as l_2 regularization in machine learning terminology. We import *Ridge* from *sklearn.linear_model* and apply this method to our problem with $\alpha = 10^{-4}$. What it does is $\min_c ||Ac - d||^2 + \alpha ||c||^2$, i.e., minimize the error by adding squares of intensities multiplied by a regularization factor, α .

In the Fig. 4.12 showing list of significant signal strengths we see that this method fails terribly to achieve right image with reasonable resolution. This is because it tries to smoothen the intensities across the sky and does not maintain sparsity of our DPH.

```

50 9.593095040406261
512 9.146548775812406
52 3.2120795574039893
48 2.817635366239541
300 2.7026941071035426
71 2.544500280394008
35 2.4987968737592285
491 2.394845742397224
253 2.3821899407021045
100 2.268919862515925

```

Figure 4.12: List of significant signal strengths with their sources. Ridge method completely fails to maintain the sparsity of the sky.

For $q=1$, $E(\vec{w})$ is called **LASSO (Least Absolute Shrinkage and Selection Operator)** or the l_1 regularization method. We import *Lasso* from *sklearn.linear_model*. We apply this method to our problem; $\min_c ||Ac - d||^2 + \alpha ||c||$ first with $\alpha = 10^{-3}$ and then with $\alpha = 10^{-2}$. In the Fig. 4.13 we see that for lower value of α , the intensities from sources other than the true ones ($i = 50, 512$) are of the order 10^{-3} . Though they are lower than those of NNLS and Ridge method, they still cannot be ignored. Whereas, in Fig. 4.14 we see that for higher value of α the method successfully achieves the sparsity nature of the sky within the same number of iterations, but compensates this with the tiny loss in accuracy of the intensities of true sources. In fact we notice that the results for the NNLS method is most accurate to the sky intensity of the true sources (refer Fig. 4.11).

```

50 29.95201097737991
512 14.989680957828403
114 0.007437973200629287
300 0.006703314028994854
197 0.005640588993824463
543 0.005015852336305533
445 0.004559669488272696
441 0.0045566646170627
828 0.004239706926552975
472 0.0025720065268551515

```

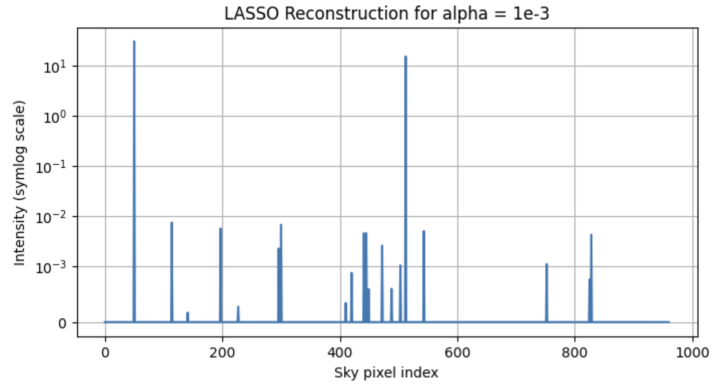
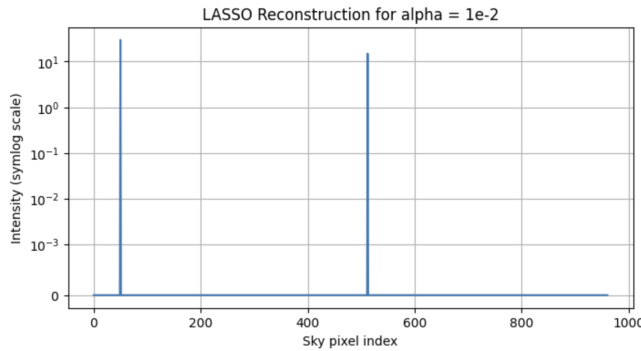


Figure 4.13: Left: List of signal strengths in decreasing order. The first column lists out the sources and the second one their respective strengths. Right: Plot of intensity v/s sky location



```

50 29.743811498595992
512 14.940204785240999
328 0.0
327 0.0
326 0.0
325 0.0
324 0.0
323 0.0
322 0.0
321 0.0

```

Figure 4.14: Left: Plot showing intensity v/s sky source. Right: List of sky intensities

We need to understand why LASSO being a linear regularizer is able to shrink the parameters of our linear model to zero which Ridge cannot do being a quadratic one. We first note that minimizing the error function (4.13) is equivalent to minimizing the unregularized sum-of-squares error function subject to the constraint

$$\sum_{j=1}^M |w_j|^q \leq \eta \quad (4.14)$$

for an appropriate value of η , where the two approaches can be related using Lagrange multipliers. The origin of sparsity can be seen in the Fig. 4.15 which shows that the minimum of the error function, subject to the constraint (4.14). As λ is increased, so an increasing number of parameters are driven to zero.

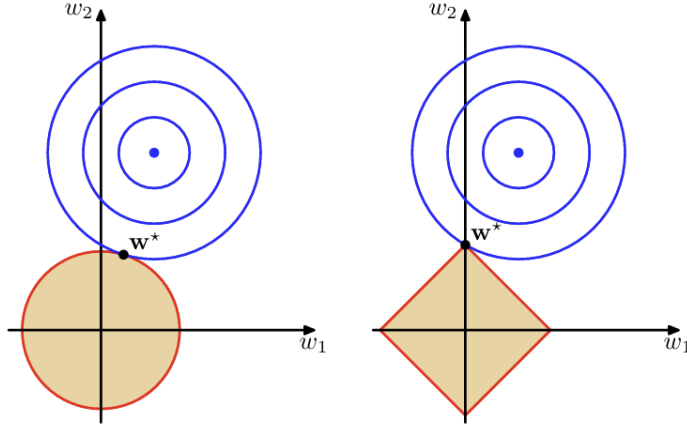


Figure 4.15: Plot of the contours of the unregularized error function (blue) along with the constraint region (3.30) for the quadratic regularizer $q = 2$ on the left and the lasso regularizer $q = 1$ on the right, in which the optimum value for the parameter vector $\vec{w}$ is denoted by $\vec{w}^*$. The lasso gives a sparse solution in which $w_1^* = 0$.

4.2.4 Sequential Learning

Determining the entire linear model in one go can be a computationally heavy task. So we update the model by considering the data points one at a time, the model parameters are updated after each iteration. We use sequential algorithms, also known as one-line algorithms.

We apply the technique of **Stochastic/Sequential Gradient Descent (SGD)**. If the error function comprises a sum over data points, $E = \sum_n E_n$, then after presentation of pattern n , the SGD algorithm updates the parameter vector $\vec{w}$ using

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} - \eta \vec{\nabla} E_n \quad (4.15)$$

where τ denotes the iteration number, and η is a learning rate parameter. Initially $\vec{w} = \vec{w}^{(0)}$. For the case of sum-of-squares error function, this gives

$$\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} - \eta (t_n - \vec{w}^{(\tau)T} \vec{\phi}_n) \vec{\phi}_n \quad (4.16)$$

where $\vec{\phi}_n = \vec{\phi}(\vec{x}_n)$. This is known as the least-mean-squares or the LMS algorithm. The value of η needs to be chosen with care to ensure that the algorithm converges.

Notice: Note that the substitution in equation (4.16) has been made in the following manner: From equation (4.6) we can write y as linear combination of fixed non linear functions of

the input variables $\vec{x}$, of the form

$$y(\vec{x}, \vec{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\vec{x}) \quad (4.17)$$

where $\phi_j(\vec{x})$ are known as basis functions. It is often convenient to define an additional dummy basis function $\phi_0(\vec{x}) = 1$ so that

$$y(\vec{x}, \vec{w}) = \sum_{j=0}^{M-1} w_j \phi_j(\vec{x}) = \vec{w}^T \cdot \vec{\phi}(\vec{x}) \quad (4.18)$$

where $\vec{w} = (w_0, w_1, \dots, w_{M-1})^T$ and $\vec{\phi} = (\phi_0, \phi_1, \dots, \phi_{M-1})^T$ are column matrices. We can substitute E in (4.15) by the form of y described in (4.18) using the relation (4.7) to get (4.16).

4.2.5 Artificial Neural Networks

Feed-forward network function[3]

The linear models for regression and discussed in sections (4.2.2) and (4.2.3), respectively, are based on linear combinations of fixed nonlinear basis functions $\phi_j(\vec{x})$ and take the form

$$y(\vec{x}, \vec{w}) = f\left(\sum_{j=1}^M w_j \phi_j(x)\right) \quad (4.19)$$

where $f(\cdot)$ is the identity in the case of regression. Our goal is to extend this model by making the basis functions $\phi_j(\vec{x})$ depend on parameters and then to allow these parameters to be adjusted, along with the coefficients $\{w_j\}$, during training. There are, of course, many ways to construct parametric nonlinear basis functions. Neural networks use basis functions that follow the same form as (4.19), so that each basis function is itself a nonlinear function of a linear combination of the inputs, where the coefficients in the linear combination are adaptive parameters.

This leads to the basic neural network model, which can be described a series of functional transformations. First we construct M linear combinations of the input variables $x_1, \dots, x_D$ in the form

$$a_j = \sum_{i=1}^D w_{ji}^{(1)} x_i + w_{j0}^{(1)} \quad (4.20)$$

where $j = 1, \dots, M$, and the superscript (1) indicates that the corresponding parameters are in the first ‘layer’ of the network. We shall refer to the parameters $w_{ji}^{(1)}$ as weights and the parameters $w_{j0}^{(1)}$ as biases. The quantities a_j are known as activations. Each of them is then transformed using a differentiable, nonlinear activation function $h(\cdot)$ to give

$$z_j = h(a_j) \quad (4.21)$$

These quantities correspond to the outputs of the basis functions in (4.19) that, in the context of neural networks, are called hidden units. The nonlinear functions $h(\cdot)$ are generally chosen to be sigmoidal functions such as the logistic sigmoid or the ‘tanh’ function. Following (4.19), these values are again linearly combined to give *output unit activations*

$$a_k = \sum_{j=1}^M w_{kj}^{(2)} z_j + w_{k0}^{(2)} \quad (4.22)$$

where $k = 1, \dots, K$, and K is the total number of outputs. This transformation corresponds to the second layer of the network, and again the $w_{k0}^{(2)}$ are bias parameters. Finally, the output unit activations are transformed using an appropriate activation function to give a set of network outputs y_k . The choice of activation function is determined by the nature of the data and the assumed distribution of target variables and follows the same considerations as for linear models discussed in sections 4.2.2 and 4.2.3. Thus for standard regression problems, the activation function is the identity so that $y_k = a_k$. Similarly, for multiple binary classification problems, each output unit activation is transformed using a logistic sigmoid function so that

$$y_k = \sigma(a_k) \quad (4.23)$$

where

$$\sigma(a) = \frac{1}{1 + \exp(-a)} \quad (4.24)$$

We can combine these various stages to give the overall network function that, for sigmoidal output unit activation functions, takes the form

$$y_k(\vec{x}, \vec{w}) = \sigma \left(\sum_{j=1}^M w_{kj}^{(2)} h \left(\sum_{i=1}^M w_{ji}^{(1)} x_i + w_{j0}^{(1)} \right) + w_{k0}^{(2)} \right) \quad (4.25)$$

where the set of all weight and bias parameters have been grouped together into a vector $\vec{w}$. *In order to prevent the neural network of our brain from getting disrupted we'd not like to go further into the mathematical modelling. Thus the neural network (NN) model is simply a nonlinear function from a set of input variables x_i to a set of output variables y_k controlled by a vector $\vec{w}$ of adjustable parameters.*

Building and training an NN model

We have 961 shadow pattern arrays each corresponding to sky locations of unit intensity. We consider each shadow array as a DPH made by its corresponding sky source of unit intensity. So our input data are these DPHs residing in space $\mathbb{R}^{256}$ and output data are the image arrays which have 1 only at their respective sky location i and rest of the elements as 0, residing in $\mathbb{R}^{961}$. Our goal is then to build a model which can find a mapping $f : \mathbb{R}^{256} \rightarrow \mathbb{R}^{961}$. This model

would go through each of these DPHs several times knowing what their output images are and try to build an algorithm to find a relationship between them.

We arrange our DPHs into a matrix S of dimension 256x961, i.e, arranging them column wise. Now what the model will try to do mathematically is

$$\vec{W}.\vec{S}_i \simeq \vec{e}_i \quad (4.26)$$

So,

$$\vec{W}.\vec{S} \simeq \vec{I} \quad (4.27)$$

$$\vec{W} \simeq S^{-1} \quad (4.28)$$

find an operator W which when operated on S gives an identity matrix I of dimension 961x961. So basically it computes the *Moore-Penrose pseudo-inverse* of $\vec{S}$.

$$S^{-1} \equiv (S^T S)^{-1} S^T \quad (4.29)$$

It then develops a linear relationship between the output and the input

$$\text{output} = \text{input}.W^T + b \quad (4.30)$$

where output is an image and input is its corresponding DPH. b is the bias vector that contributes to noise. The model used to fit the data is decided by the NN model itself. So now we'll build and test a NN model in *Tensorflow* engine through the following steps:

Step1: Building training set

We build an input and an output variable matrix X and Y respectively. Instead of assigning Y as an identity matrix of order 961x961, we fill its diagonal elements with random numbers from 1 to 100. We do this because in the equation (4.28) there is a scale factor which equates both the sides. So when the machine tries to predict the image intensity at a certain location which is unity, it multiplies it with the scale factor. To avoid this I need to train it on random intensities. In short

$$Y[i, i] = \text{amp}, \text{ where amp is some random number} \quad (4.31)$$

$$X[i] = \text{amp} * \text{shadow}[i] \quad (4.32)$$

where shadow[i] is the ith shadow array in the shadow matrix.

Step2: Building the Neural Network

We build a model by which the output is connected to the input through a sequence of layers by *keras.Sequential* model. We define our input to be a matrix of order 1x256 and output to be of the order 1x961 and then join both of them by a single *Dense* layer. This layer allows every

output neurons to use all the 256 values. In order to avoid getting negative sky intensities we use the "softplus" tool with l_1 regularization set to $1e-4$.

Step3: Compiling the model

We use *keras.optimizers* to run the iterations using *Adam* (Adaptive Moment Estimation) which is a type of intelligent gradient descent (refer section 4.2.4). We set the learning rate to 10^{-3} , which would allow the machine to change the model parameters (W and b) by this amount to fit the data into the relationship defined by equation (4.30). In each iteration it computes the loss which is the mean square error or variance of the predicted image and tries to minimize it.

$$\text{loss} = \frac{1}{961} \sum_i |y_i - \hat{y}_i|^2 \quad (4.33)$$

Step4: Training the network

Now we fit the input X and output Y . The network goes through all the 961 DPHs 500 times (epochs) to find relationship with their images. In each go it divides the number of DPHs into batch sizes of 32, with a total of approximately 31 batches. The network adjusts W and b to minimize the error. We can see the time taken by it to learn through a single batch and the loss value in each epoch while it is being trained.

Step5: Testing on synthetic DPHs

We now create few synthetic DPHs and run the model to predict their images. The result has been shown in the figure below. We notice that the model is able to successfully predict the exact position of the sources across the sky in all of the DPHs. But it fails to determine their intensities by a large margin. However, it does a remarkable job in maintaining the sparsity of the sky, where other methods like NNLS and Ridge fail.

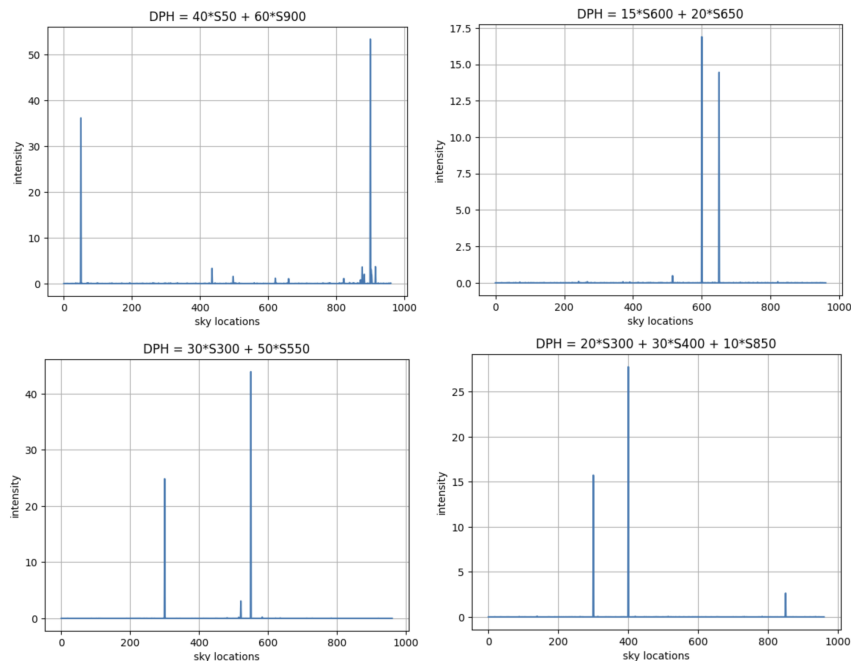


Figure 4.16

Conclusion

The most widely used method in γ -ray imaging is the Richard-Lucy Bayesian iteration method. However, due to this method being computationally heavy and unable to predict image in real time, there is a demand in the field of high energy astronomy to search for more efficient and faster methods. We made an effort to reconstruct the image using machine learning algorithms from a given DPH. We used three methods: NNLS, Ridge and LASSO. The NNLS method gave a good image reconstruction for a clean DPH, but failed to maintain the sparsity of the sky when the DPH was noisy. The Ridge method was neither able to achieve right intensity of sky distribution nor maintain its sparsity. Rather it smoothens out the sky intensity in the vicinity of the source by equally distributing it among other sources near the true source. So we notice that the right method depends upon the type of DPH. Since, our DPH being a sparse one and not uniformly distributed, LASSO was the best method to reconstruct the image. It gave satisfactorily accurate intensity distribution along with maintaining the sparsity of the sky when a suitable regularization factor was used. However, the NNLS method beats LASSO in achieving the accuracy of the intensity distribution by a slight margin. A better way would be to implement LASSO first to know where the sources are in the sky and then do the NNLS method to get most accurate image.

All these methods try to fit a model into the data to predict the image by doing stochastic gradient descent. But to get the most of machine learning we tried to build a neural network model, which could predict the image as soon as it receives the data. Our model has a single dense layer, we train it several times with our input and output data. However, at the end of the day we could not make it a perfect one as it is unable to determine the intensity distribution of the sky for any DPH. But there is a ray of hope as it can still predict the exact locations of the sources and is also able to achieve the sparsity nature of the sky.

So, making a perfect nn model is still an open problem for us. We are still working on the model and trying every way possible to think and understand what is going on inside its artificial mind to make it learn better and faster to produce good results. Our progress is slow but promising and we might not come up with a perfect model soon, but we will some day. To keep a track on our progress, follow the author on github (though he does not guarantee for frequent updates).

Bibliography

- [1] J. Braga, “Coded aperture imaging in high-energy astrophysics,” *Publications of the Astronomical Society of the Pacific*, vol. 132, p. 012001, Dec. 2019.
- [2] A. Vibhute, D. Bhattacharya, N. P. S. Mithun, V. Bhalerao, A. R. Rao, and S. V. Vadawale, “Imaging calibration of astrosat cadmium zinc telluride imager (czti),” *Journal of Astrophysics and Astronomy*, vol. 42, July 2021.
- [3] C. M. Bishop, *Pattern recognition and machine learning*. Springer, Jan. 2006.
- [4] A. Géron, *Hands-On Machine Learning with Scikit-Learn and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems*. Mar. 2017.
- [5] J. Vanderplas, *Python data science handbook essential tools for working with data*. Jan. 2016.
- [6] W. McKinney, *Python for data analysis data wrangling with Pandas, NumPy, and IPython*. Sept. 2018.
- [?]